

PZ-ESP8266 WIFI 模块开发手册

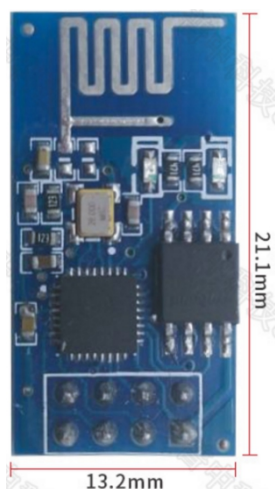
1 PZ-ESP8266 模块介绍与测试

1.1 ESP8266 简介

ESP8266 是一个完整且成体系的 Wi-Fi 网络解决方案，能够搭载软件应用，或通过另一个应处理器卸载所有 Wi-Fi 网络功能。我们使用的 ESP8266 是串口型 WIFI，速度比较低，不能用来传输图像或者视频这些大容量的数据，主要应用于数据量传输比较少的场合，比如温度信息，一些传感器的开关量等。当然传输的数据量虽说少，但也能一次传输几千字节的数据，而且通信非常稳定，可以满足大多数应用。

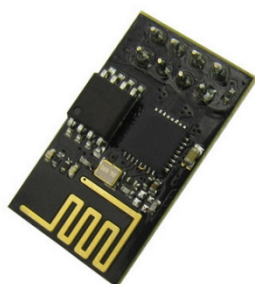
1.2 PZ-ESP8266 模块介绍

安信可推出的 ESP8266-WIFI 模块有很多，它们的使用方法都大同小异，我们普中 STM32 开发板预留了一个 ESP8266-WIFI 模块接口，大家只要将此模块插入开发板接口即可做此实验，无需额外连线。我们普中推出了 1 款 WIFI 模块，如图：

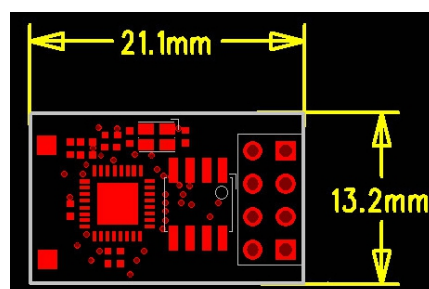


与通用的 WIFI 模块一样使用，这里我们就以通用的 WIFI 模块介绍。如下图

所示：

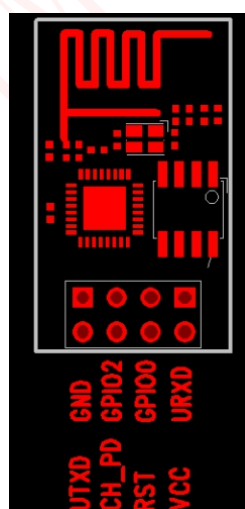


上图的 WIFI 模块尺寸图如下图所示：



如果要将此模块设计到自己产品内，可能需要参考这个尺寸值。

从 WIFI 模块实物图中可以看到，WIFI 模块提供了一个 2*4 的外接管脚，让我们连接到自己的电路中控制，这 8 个管脚两两间距是 2.54mm。管脚功能定义如下：



VCC: 3.3V 电源，开发板上丝印已经标了。

RST: ES8266 复位管脚，可做外部硬件复位使用。

CH_PD: 使能管脚，高电平有效。

UTXD: 串口发送管脚，与开发板上串口 3 的 RXD 相连。

URXD: 串口接收管脚，与开发板上串口 3 的 TXD 相连。

GPI00: GPI00 为高电平代表从 FLASH 启动，GPI00 为低电平代表进入系统升级状态，此时可以经过串口升级内部固件，这里我们不需要对此管脚操作。

GPI02: 此管脚为 ESP8266 引出的一个 IO 口，这里我们不需要对此管脚操作。

GND: GND 管脚，开发板上丝印已经标了。

其实我们不需要使用这么多管脚，只需要使用 WIFI 模块的串口 UTXD、URXD 管脚、RST 和 CH_PD 管脚即可，其他的不用管。

ESP8266-WIFI 模块支持 STA/AP/STA+AP 三种工作模式。

STA 模式: ESP8266 模块通过路由器连接互联网，手机或电脑通过互联网实现对设备的远程控制。

AP 模式: 默认模式 PZ_ESP8266 模块作为热点，实现手机或电脑直接与模块通信，实现局域网无线控制。

STA+AP 模式: 两种模式的共存模式，即可以通过互联网控制可实现无缝切换，方便操作。

1.3 PZ-ESP8266 模块测试

下面开始对模块进行测试。

1.3.1 硬件准备

首先我们需要准备以下设备：

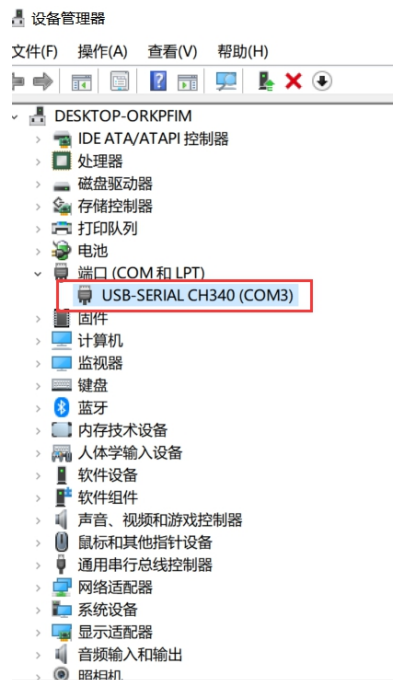
- ①PZ-ESP8266 模块一个
- ②电脑一台
- ③USB 转 TTL 模块或普中 STM32 开发板（普中所有 STM32 开发板均含有 USB 转 TTL 模块）一块
- ④USB 数据线

1.3.2 使用 USB 转 TTL 串口测试

使用“\3--软件工具\串口调试助手\XCOM（推荐）\XCOM V2.0.exe”可方便地测试 PZ-ESP8266 模块是否正常，测试步骤如下：

(1) 安装 USB 转 TTL 模块驱动

使用一根 USB 数据线连接 USB 转 TTL 模块或普中 STM32 开发板, 如果使用普中 STM32 开发板用户, 可直接找到 CH340 驱动安装, 这里默认已经安装好了 CH340 驱动, 此时计算机就可识别开发板的 CH340 串口, 可在设备管理器内查看到, 如下图所示:



(2) 检测模块是否正常

确保驱动安装成功后, 将 PZ-ESP8266 模块与开发板上 USB 转 TTL 模块按照如下连接: (PZ-ESP8266 模块-->STM32 开发板上 USB 转 TTL 模块) (**事先将开发板上 USB 转 TTL 模块 P4 端子的黄色跳线帽取下**)



VCC-->3.3V

CH_PD-->3.3V

TXD-->URXD (开发板上 USB 转 TTL 模块)

RXD-->UTXD（开发板上 USB 转 TTL 模块）

GND-->GND

连接好模块和开发板之间的引脚，再用 USB 数据线连接好电脑和开发板，给开发板上电。打开“**XCOM V2.0.exe**”，在端口菜单中选择步骤设备管理器中识别的 CH340 串口号，选择默认波特率“9600”，打开串口，勾选“发送新行”选项。在“字符串输入框”输入“AT”，点击“发送”，如果上方接收窗口有返回“OK”，那说明模块正常。如果没有返回“OK”，可以多次尝试，或检查前面步骤是否正常，**如若仍无该返回，请按照教程后面章节升级下固件**。升级后仍然无返回，模块可能存在异常。如下图所示：



注意：如果模块没有返回 OK，可以选择重新烧写固件测试，这个在后面小节会有介绍。

1.3.3 通过检测模块 WIFI 信号

（1）硬件准备

该测试前需要准备以下一些设备：

- ①PZ-ESP8266 模块一个

②能连接 WiFi 的手机、PAD 或笔记本等一部（我使用的手机）

③普中任意一款 STM32 开发板（作为 3.3V 供电）

（2）进入测试

该测试方法非常简单，只是检测模块是否能发出 WiFi 信号。

①按照下列方式连接好电路（PZ-ESP8266 模块-->普中 STM32 开发板）

VCC-->3.3V

GND-->GND

CH_PD-->3.3V

②使用手机扫描 WiFi 信号

给模块上电，打开手机的“WLAN”，扫描周围的热点，如果能扫描到模块发出的热点，如下图所示，那说明模块基本正常。该热点一般是“OPEN”型的，也就是无密码的，用户可连连看。模块断电后，该热点就消失。



2 串口 AT 指令

2.1 概述

指令集主要分为：基础 AT 命令、Wifi 功能 AT 命令、TCP/IP 工具箱 AT 命令等。

2.2 指令说明

每条指令可以有细分四种命令形式。

命令形式	AT 指令格式	说明
测试命令	AT+<x>=?	该命令用于查询设置命令或内部程序设置的参数以及其取值范围。
查询命令	AT+<x>?	该命令用于返回参数的当前值。
设置命令	AT+<x>=<...>	该命令用于设置用户自定义的参数值。
执行命令	AT+<x>	该命令用于执行受模块内部程序控制的变参数不可变的功能。

- 注意：1. 不是每条 AT 指令都具备上述 4 类命令
2. []内数据为缺省值，不必填写或可能不显示
3. 使用双引号表示字符串数据
4. 默认波特率 960000

2.3 基础 AT 指令

2.3.1 基础 AT 指令一览表

命令	说明
AT	测试 AT 启动
AT+RST	重启模块
AT+GMR	查看版本信息

2.3.2 基础 AT 指令详解

1、AT：测试 AT 启动

AT	说明
执行指令 AT	响应 OK

2、AT+RST：重启模块

AT+RST	说明
执行指令 AT+RST	响应 OK

3、AT+GMR：查看版本信息

AT+GMR	说明
执行指令 AT+GMR	响应 <number> OK
	参数说明 < number >8 位版本号

2.4 Wifi 功能 AT 指令

2.4.1 Wifi 功能 AT 指令一览表

命令	说明
AT+CWMODE	选择 WIFI 应用模式
AT+CWJAP	加入 AP
AT+CWLAP	列出当前可用 AP
AT+CWQAP	退出与 AP 的连接
AT+CWSAP	设置 AP 模式下的参数
AT+CWLIF	查看已接入设备的 IP

2.4.2 Wifi 功能 AT 指令详解

1、AT+CWMODE：选择 WIFI 应用模式

AT+CWMODE	说明
测试指令 AT+CWMODE=?	响应 +CWMODE:(<mode>取值列表) OK
	参数说明 见设置命令
查询命令 AT+CWMODE?	响应 返回当前模块的模式 +CWMODE:<mode> OK
	参数说明 见设置指令

设置指令 AT+CWMODE=<mode>	响应 OK
	参数说明 <mode>1 Station 模式 2 AP 模式 3 AP 兼 Station 模式

2、AT+CWJAP：加入 AP

AT+CWJAP	说明
查询命令 AT+CWJAP?	响应 返回当前选择的 AP + CWJAP:<ssid> OK
	参数说明 见设置命令
设置指令 AT+CWJAP = <ssid>,<pwd>	响应 OK ERROR
	参数说明 <ssid>字符串参数，接入点名称 <pwd>字符串参数，密码最长 64 字节 ASCII

3、AT+CWLAP：列出当前可用 AP

AT+CWLAP	说明
执行指令 AT+CWLAP	响应 终端返回 AP 列表 + CWLAP: <ecn>,<ssid>,<rssi> OK ERROR
	参数说明 <ecn>0 OPEN 1 WEP 2 WPA_PSK 3 WPA2_PSK 4 WPA_WPA2_PSK <ssid>字符串参数，接入点名称 <rssi>信号强度

4、AT+CWQAP：退出与 AP 的连接

AT+CWQAP	说明
测试指令 AT+CWQAP=?	响应 OK
执行指令 AT+CWQAP	响应 OK

5、AT+CWSAP：设置 AP 模式下的参数

AT+CWSAP	说明
查询命令 AT+ CWSAP?	响应 返回当前 AP 参数 + CWSAP:<ssid>,<pwd>,<chl>,<ecn> 参数说明 见设置指令
设置指令 AT+ CWSAP= <ssid>,<pwd>,<chl>,<ecn>	响应 OK ERROR 参数说明 指令只有在 AP 模式开启后有效 <ssid>字符串参数，接入点名称 <pwd>字符串参数，密码最长 64 字节 ASCII <chl>通道号 <ecn>0 OPEN 1 WEP 2 WPA_PSK 3 WPA2_PSK 4 WPA_WPA2_PSK

6、AT+CWLIF：查看已接入设备的 IP

AT+CWLIF	说明
执行指令 AT+CWLIF	响应 <ip addr> OK 参数说明 <ip addr> 已接入设备的 IP 地址

2.5 TCP/IP 工具箱 AT 指令

2.5.1 TCP/IP 工具箱 AT 指令一览表

命令	说明
AT+ CIPSTATUS	获得连接状态
AT+CIPSTART	建立 TCP 连接或注册 UDP 端口号
AT+CIPSEND	发送数据
AT+CIPCLOSE	关闭 TCP 或 UDP
AT+CIFSR	获取本地 IP 地址
AT+CIPMUX	启动多连接
AT+CIPSERVER	配置为服务器
AT+CIPMODE	设置模块传输模式
AT+CIPSTO	设置服务器超时时间

2.5.2 TCP/IP 工具箱 AT 指令详解

1、AT+CIPSTATUS：获得连接状态

AT+CIPSTATUS	说明
测试指令 AT+ CIPSTATUS=?	响应 OK
执行指令 AT+ CIPSTATUS	响应 返回当前模块的连接状态和连接参数 STATUS:<stat> + CIPSTATUS:<id>,<type>,<addr>,<port>,<tetype> OK
	参数说明 <id>连接的 id 号 0-4 <type>字符串参数，类型 TCP 或 UDP <addr>字符串参数，IP 地址 <port>端口号 <tetype> 0: 本模块做 client 的连接 1: 本模块做 server 的连接

2、AT+CIPSTART：建立 TCP 连接或注册 UDP 端口号

AT+CIPSTART	说明
测试指令 AT+CIPSTART=?	响应 1) 设置 AT+CIPMUX=0 +CIPSTART:(<type>取值列表),(<IP address>范围),(<port>范围) +CIPSTART:(<type>取值列表),(<domain name>范围),(<port>范围) OK 2) 设置 AT+CIPMUX=1 +CIPSTART:(id),(<type>取值列表),(<IP address>范围),(<port>范围) +CIPSTART: (id), (<type>取值列表),(<domain name>范围),(<port>范围)
	参数说明 见设置命令
设置命令 1)单路连接 (+CIPMUX=0) AT+CIPSTART= <type>,<addr>,<port>	响应 如果格式正确且连接成功，返回 OK 否则返回 ERROR 如果连接已经存在，返回

1)多路连接 (+CIPMUX=1) AT+CIPSTART= <id><type>,<addr>, <port>	ALREAY CONNECT
	参数说明 <id>连接的 id 号 0-4 <type> 字符串参数, 表明连接类型 " TCP" 建立 tcp 连接 " UDP" 建立 UDP 连接 <addr> 字符串参数, 远程服务器 IP 地址 <port> 远程服务器端口号

3、AT+CIPSEND: 发送数据

AT+CIPSEND	说明
测试指令 AT+CIPSEND=?	响应 OK 参数说明 见设置命令
设置指令 1) 单路连接时 (+CIPMUX=0) AT+CIPSEND=<length> 2) 多路连接时 (+CIPMUX=1) AT+CIPSEND= <id>,<length>	响应 发送指定长度的数据。收到此命令后先换行返回">", 然后开始接收串口数据, 当数据长度满 length 时发送数据。 如果未建立连接或连接被断开, 返回 ERROR 如果数据发送成功, 返回 SEND OK 参数说明 <id>需要用于传输连接的 id 号 <length>数字参数, 表明发送数据的长度, 最大长度为 2048
执行指令 AT+CIPSEND	响应 收到此命令后先换行返回">" 然后就进入了透传模式, 每包数据以 20ms 间隔区分, 每包最大 2048 字节。 当输入单独一包"+++" 返回指令模式。 该指令必须在开启透传模式以及单连接模式下使用

4、AT+CIPCLOSE: 关闭 TCP 或 UDP

AT+CIPCLOSE	说明
测试指令 AT+CIPCLOSE=?	响应 OK
多路连接时 AT+CIPCLOSE=<id>	如果输入正确, 返回 OK 如果没有该连接则, 返回 Link is not 参数说明 <id>需要关闭的连接 id

	当 id=5 时关闭所有连接（开启 server 后 id=5 无效）
执行指令 单路连接时 AT+CIPCLOSE	响应 如果输入正确，返回 OK 如果没有连接则，返回 ERROR 当没有连接时返回状态打印 unlink

5、AT+CIFSR：获取本地 IP 地址

AT+CIFSR	说明
测试指令 AT+CIFSR=?	响应 OK
执行命令 AT+ CIFSR	响应 + CIFSR:<AP IP address> <STA IP address> OK ERROR
	参数说明 <AP IP address> 本机的 AP 模式的 IP 地址 <STA IP address> 本机的 STA 模式的 IP 地址
参考	说明 对应应用模式开通了才有其 IP 地址

6、AT+CIPMUX：启动多连接

AT+CIPMUX	说明
查询命令 AT+ CIPMUX?	响应 + CIPMUX:<mode> OK
	参数说明 见设置指令
设置指令 AT+ CIPMUX=<mode>	响应 OK 如果已经处于连接状态则，返回 Link is builded
	参数说明 <mode>0 单路连接模式 1 多路连接模式
参考	说明 只有当连接都断开后才能更改，如果开启过 server 需要重启模块

7、AT+CIPSERVER：配置为服务器

设置指令 AT+ CIPSERVER= <mode>[,<port>]	响应 OK 关闭 server 需要重启
	参数说明 <mode>0 关闭 server 模式 1 开启 server 模式 <port>端口号, 缺省值为 333
参考	说明 开启 server 后自动建立 server 监听 当有 client 接入会自动按顺序占用一个连接 AT+ CIPMUX=1 时才能开启服务器

8、AT+CIPMOD：设置模块传输模式

AT+CIPMOD	说明
查询命令 AT+ CIPMODE?	响应 + CIPMODE:<mode> OK
	参数说明 见设置指令
设置指令 AT+CIPMODE=<mode>	响应 OK 如果已经处于连接状态则, 返回 Link is builded
	参数说明 <mode>0 非透传模式 1 透传模式

9、AT+CIPSTO：设置服务器超时时间

AT+CIPSTO	说明
查询命令 AT+CIPSTO?	响应 + CIPSTO:<time> OK
	参数说明 见设置指令
设置指令 AT+CIPSTO=<time>	响应 OK
	参数说明 < time>0~28800 服务器超时时间, 单位为 s

2.6 其他指令

1、+IPD：接收到网络数据

+IPD	说明
参考 1) 单路连接时 (+CIPMUX=0) +IPD,<len>:<data> 2) 多路连接时 (+CIPMUX=1) +IPD,<id>,<len>:<data>	说明 此指令是模块发出指令，当模块接收到网络数据时向串口发送+IPD 和数据 <id>收到连接的 id 号 <len>数据长度 <data>收到的数据 此提示在指令状态下有效

2、AT+CIOBAUD：设置波特率

AT+CIOBAUD	说明
测试指令 AT+CIOBAUD=?	响应 +CIOBAUD:<baud_range> OK 参数说明 <baud_range>波特率取值范围
查询命令 AT+CIOBAUD?	响应 +CIOBAUD:<baud> OK 参数说明 <baud>当前的波特率
设置指令 AT+CIOBAUD=<baud>	响应 BAUD-><baud> OK 参数说明 <baud>要设置的波特率

2.7 附带说明

上述 AT 指令集是 PZ-ESP8266 模块的现有的常用 AT 指令集，模块的具有 AT 指令集因固件不同而不同，更新版本的固件一般包含更丰富的 AT 指令，并且一般新固件都兼容就固件。用户若想刷新版固件，可参考后面章节。

3 STM32 控制 PZ-ESP8266 模块

PZ-ESP8266 模块支持 TTL 串口通讯标准，可直接与单片机的串口引脚连接通讯，非常方便使用单片机系统来控制。本章节以普中 STM32 开发板为例说明如何使用 STM32 来控制 PZ-ESP8266 模块。

3.1 硬件连接

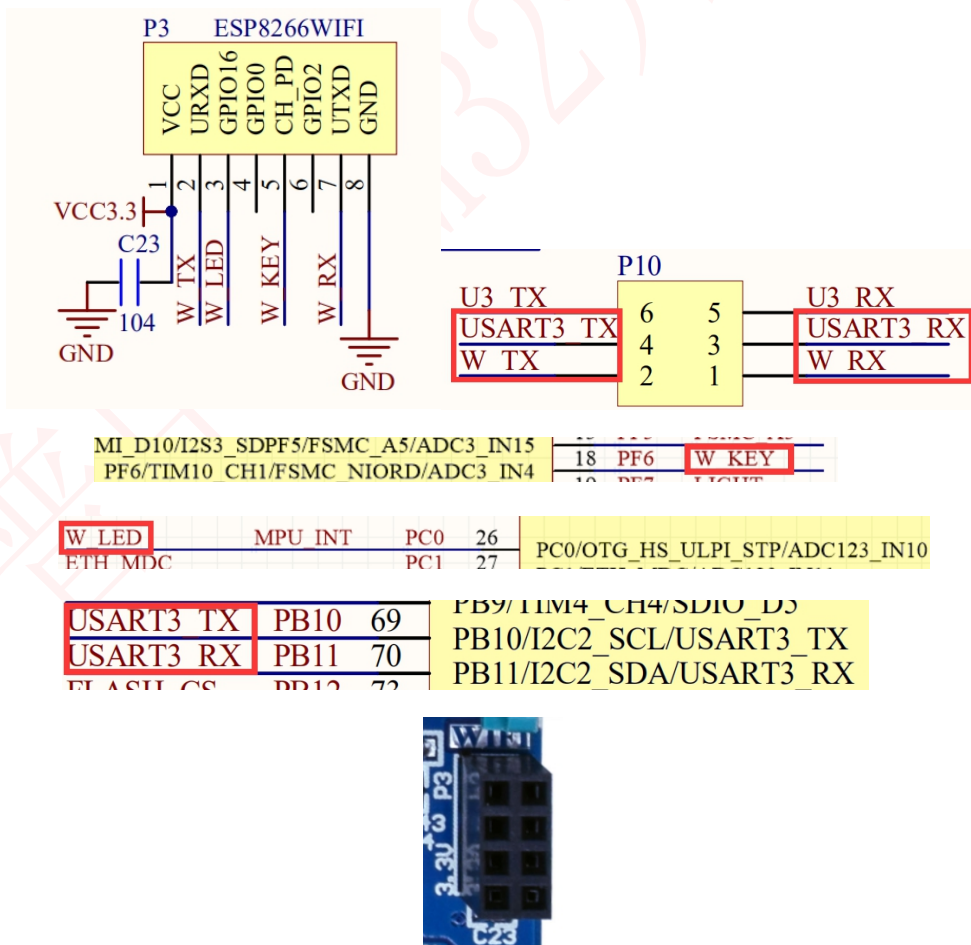
3.1.1 硬件准备

本实验所需要的硬件资源如下：

- ①F407 最小系统&天马&麒麟开发板 1 个
- ②PZ-ESP8266 WIFI 模块 1 个
- ③USB 线一条（用于供电和模块与电脑串口调试助手通信）
- ④安卓手机一台

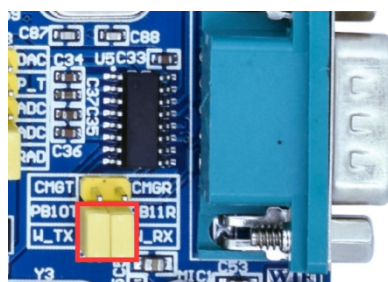
3.1.2 模块与开发板连接

PZ-ESP8266 模块可直接与天马&麒麟开发板板载的 WIFI&蓝牙模块接口 (P3) 进行连接, 该接口与 MCU 连接原理图 (以麒麟为例) 如下图所示:



从上图看出，WIFI 接口与 STM32 使用时，必须将 P10 的 USART3_TX (PB10

和 W_TX 以及 USART3_RX (PB11) 和 W_RX 连接, 才能完成和 STM32 的连接。默认天马&麒麟的 P10 端子上有黄色跳线帽, 直接短接到对应端即可。如下图所示:



对于 F407 最小系统, 是没有 WIFI 接口的, 所以需要通过杜邦线将模块管脚与上述 STM32 的 IO 口连接即可。

PZ-ESP8266 模块与 STM32 的 IO 口连接关系如下:(PZ-ESP8266 模块-->STM32 开发板)

```
VCC-->3.3V
RST(I016)-->PC0
CH_PD-->PF6
TXD-->PB11 (USART3)
RXD-->PB10 (USART3)
GND-->GND
```

注意: 使用天马&麒麟开发板连接模块时, 模块插入 WIFI 接口处请注意方向, 可将模块电源与 WIFI 接口丝印电源对应即可。

3.2 软件设计

我们打开对应开发板型号的例程, 该程序实现的功能是: 使用串口调试助手作界面来配置和使用 PZ-ESP8266 模块, 按照串口调试助手打印的提示配置模块, 配置完成之后控制模块与电脑/手机进行数据传输。

下面开始分析下程序, 这里我们主要讲解几个关键函数, 详细的代码大家可以打开工程查看。

程序从 main.c 文件的主函数 main 开始运行, 代码如下, 根据用户的输入对 PZ-ESP8266 WiFi 模块选择不同的测试模式, 然后调用相应的测试函数。

```
1.  int main()
2.  {
```

```

3.     char cCh;
4.
5.     SysTick_Init(168);
6.     NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2); //中断优先级分组 分2组
7.     USART1_Init(115200);
8.     LED_Init();
9.     WiFi_Config();
10.
11.    PC_Usart("\r\n 普中 PZ-ESP8266 WiFi 模块测试例程\r\n"); //打印测试例程提示信息
12.    PC_Usart ( "\r\n 请输入模块的测试模式的编号，编号如下： \
13.                \r\n1.STA 模式： TCP Client\
14.                \r\n2.AP 模式： AP Server\
15.                \r\n3.STA+AP 模式： (STA)TCP Client + (AP)TCP Server\
16.                \r\n 输入格式为： 编号+空格，点击发送\r\n" );
17.
18.    scanf("%c", &cCh );
19.    switch(cCh)
20.    {
21.        case '1':
22.            pNet_Test = ESP8266_STA_TCP_Client;
23.            PC_Usart("\r\n 即将进行的测试是： 1.STA 模式： TCP Client\r\n");
24.            break;
25.
26.        case '2':
27.            pNet_Test = ESP8266_AP_TCP_Server;
28.            PC_Usart("\r\n 即将进行的测试是： 2.AP 模式： TCP Server\r\n");
29.            break;
30.
31.        case '3':
32.            pNet_Test = ESP8266_StaTcpClient_ApTcpServer;
33.            PC_Usart("\r\n 即将进行的测试是： 3.STA+AP 模
式： (STA)TCP Client + (AP)TCP Server\r\n");
34.            break;
35.
36.        default:
37.            pNet_Test = NULL;
38.            PC_Usart("\r\n 输入错误，不进行测试！ \r\n");
39.            break;
40.
41.    }
42.    if(pNet_Test)
43.        pNet_Test();
44.
45.    while(1);

```

46. }

在 main 函数中选择了测试模式后，程序就跳至 wifi_function.c 文件执行相关测试模式函数。如果选择了“STA”模式，就执行 ESP8266_STA_TCP_Client 函数，以模块作为 TCP 客户端为例进行测试，见代码清单①；如果选择了“AP”模式，就执行 ESP8266_AP_TCP_Server 函数，以模块作为 TCP 服务器为例进行测试，见代码清单②；如果选择了“STA+AP”模式，就执行 ESP8266_StaTcpClient_ApTcpServer 函数，以模块既作为 TCP 客户端又作为 TCP 服务器为例进行测试，见代码清单③。

代码清单①：ESP8266_STA_TCP_Client 函数

```
1.  /*
2.  * 函数名: ESP8266_STA_TCP_Client
3.  * 描述  : PZ-ESP8266 模块进行 STA TCP Clie 测试
4.  * 输入  : 无
5.  * 返回  : 无
6.  * 调用  : 被外部调用
7.  */
8.  void ESP8266_STA_TCP_Client ( void )
9.  {
10.     char cStrInput [100] = { 0 }, * pStrDelimiter [2], * pBuf, * pStr;
11.     u8 uc = 0;
12.     u32 ul = 0;
13.
14.     ESP8266_Choose ( ENABLE );
15.
16.     ESP8266_AT_Test ();
17.
18.     ESP8266_Net_Mode_Choose ( STA );
19.
20.     ESP8266_Cmd ( "AT+CWLAP", "OK", 0, 5000 );
21.
22.     do
23.     {
24.         PC_Usart ( "\r\n 请输入要连接的 WiFi 名称和密钥, 输入格式为: 名称字符+英文逗号+密钥字符+空格,
           点击发送\r\n" );
25.
26.         scanf ( "%s", cStrInput );
27.
28.         PC_Usart ( "\r\n 稍等片刻 ..... \r\n" );
29.
30.         pBuf = cStrInput;
```



```

31.         uc = 0;
32.         while ( ( pStr = strtok ( pBuf, "," ) ) != NULL )
33.         {
34.             pStrDelimiter [ uc ++ ] = pStr;
35.             pBuf = NULL;
36.         }
37.
38.     } while ( ! ESP8266_JoinAP ( pStrDelimiter [0], pStrDelimiter [1] ) );
39.
40.     ESP8266_Enable_MultipleId ( ENABLE );
41.
42.     do
43.     {
44.         PC_Usart ( "\r\n 请在电脑上将网络调试助手以 TCP Server 连接网络, 并输入电脑的 IP 和端口号, 输入格式为: 电脑 IP+英文逗号+端口号+空格, 点击发送\r\n" );
45.
46.         scanf ( "%s", cStrInput );
47.
48.         PC_Usart ( "\r\n 稍等片刻 ..... \r\n" );
49.
50.         pBuf = cStrInput;
51.         uc = 0;
52.         while ( ( pStr = strtok ( pBuf, "," ) ) != NULL )
53.         {
54.             pStrDelimiter [ uc ++ ] = pStr;
55.             pBuf = NULL;
56.         }
57.
58.     } while ( ! ( ESP8266_Link_Server ( enumTCP, pStrDelimiter [0], pStrDelimiter [1], Multiple_ID_0 ) &&
59.                 ESP8266_Link_Server ( enumTCP, pStrDelimiter [0], pStrDelimiter [1], Multiple_ID_1 ) &&
60.                 ESP8266_Link_Server ( enumTCP, pStrDelimiter [0], pStrDelimiter [1], Multiple_ID_2 ) &&
61.                 ESP8266_Link_Server ( enumTCP, pStrDelimiter [0], pStrDelimiter [1], Multiple_ID_3 ) &&
62.                 ESP8266_Link_Server ( enumTCP, pStrDelimiter [0], pStrDelimiter [1], Multiple_ID_4 ) ) );
63.
64.     for ( uc = 0; uc < 5; uc ++ )
65.     {
66.         PC_Usart ( "\r\n 请输入端口 ID%d 要发送的字符串, 输入格式为: 字符串 (不含空格)+空格, 点击发送\r\n", uc );
67.

```

```

68.         scanf ( "%s", cStrInput );
69.
70.         ul = strlen ( cStrInput );
71.
72.         ESP8266_SendString ( DISABLE, cStrInput, ul, ( ENUM_ID_NO_TypeDef ) uc );
73.
74.     }
75.
76.
77.     PC_Usart ( "\r\n 请在网络调试助手发送字符串\r\n" );
78.     while (1)
79.     {
80.         pStr = ESP8266_ReceiveString ( DISABLE );
81.         PC_Usart ( "%s", pStr );
82.     }
83.
84. }

```

代码清单②：ESP8266_AP_TCP_Server 函数

```

1.  /*
2.  * 函数名: ESP8266_AP_TCP_Server
3.  * 描述   : PZ-ESP8266 模块进行 AP TCP Server 测试
4.  * 输入   : 无
5.  * 返回   : 无
6.  * 调用   : 被外部调用
7.  */
8.  void ESP8266_AP_TCP_Server ( void )
9.  {
10.     char cStrInput [100] = { 0 }, * pStrDelimiter [3], * pBuf, * pStr;
11.     u8 uc = 0;
12.     u32 ul = 0;
13.
14.     ESP8266_Choose ( ENABLE );
15.
16.     ESP8266_AT_Test ();
17.
18.     ESP8266_Net_Mode_Choose ( AP );
19.
20.
21.     PC_Usart ( "\r\n 请输入要创建的 WiFi 的名称、加密方式和密钥，加密方式的编号为: \
22.               \r\n0 = OPEN\
23.               \r\n1 = WEP\
24.               \r\n2 = WPA_PSK\
25.               \r\n3 = WPA2_PSK\
26.               \r\n4 = WPA_WPA2_PSK\

```

```

27.                                     \r\n 输入格式为: 名称字符+英文逗号+加密方式编号+英文逗号+密钥字符+空格,
   点击发送\r\n" );
28.
29.     scanf ( "%s", cStrInput );
30.
31.     PC_Usart ( "\r\n 稍等片刻 ..... \r\n" );
32.
33.     pBuf = cStrInput;
34.     uc = 0;
35.     while ( ( pStr = strtok ( pBuf, "," ) ) != NULL )
36.     {
37.         pStrDelimiter [ uc ++ ] = pStr;
38.         pBuf = NULL;
39.     }
40.
41.     ESP8266_BuildAP ( pStrDelimiter [0], pStrDelimiter [2], pStrDelimiter [1] );
42.     ESP8266_Cmd ( "AT+RST", "OK", "ready", 2500 ); /*
43.
44.
45.     ESP8266_Enable_MultipleId ( ENABLE );
46.
47.
48.     PC_Usart ( "\r\n 请输入服务器要开启的端口号和超时时间 (0~28800, 单位: 秒), 输入格式为: 端口号
   字符+英文逗号+超时时间字符+空格, 点击发送\r\n" );
49.
50.     scanf ( "%s", cStrInput );
51.
52.     PC_Usart ( "\r\n 稍等片刻 ..... \r\n" );
53.
54.     pBuf = cStrInput;
55.     uc = 0;
56.     while ( ( pStr = strtok ( pBuf, "," ) ) != NULL )
57.     {
58.         pStrDelimiter [ uc ++ ] = pStr;
59.         pBuf = NULL;
60.     }
61.
62.     ESP8266_StartOrShutServer ( ENABLE, pStrDelimiter [0], pStrDelimiter [1] );
63.
64.
65.     do
66.     {
67.         PC_Usart ( "\r\n 正查询本模块 IP..... \r\n" );
68.         ESP8266_Cmd ( "AT+CIFSR", "OK", "Link", 500 );

```

```

69.
70.         PC_Usart ( "\r\n 用手机连接刚才创建的 WiFi, 这里只连接一个手机, 作为 ID0, 然后用手机网络调
           试助手以 TCP Client 连接刚才开启的服务器 (AP IP) ..... \r\n" );
71.         delay_ms ( 20000 );
72.     } while ( ! ESP8266_Cmd ( "AT+CIPSTATUS", "+CIPSTATUS:0", 0, 500 ) );
73.
74.
75.         PC_Usart ( "\r\n 请输入要向端口手机 (ID0) 发送的字符串, 输入格式为: 字符串 (不含空格)+空格, 点
           击发送\r\n" );
76.
77.         scanf ( "%s", cStrInput );
78.
79.         ul = strlen ( cStrInput );
80.
81.         ESP8266_SendString ( DISABLE, cStrInput, ul, Multiple_ID_0 );
82.
83.
84.         PC_Usart ( "\r\n 请在手机网络调试助手发送字符串\r\n" );
85.         while (1)
86.         {
87.             pStr = ESP8266_ReceiveString ( DISABLE );
88.             PC_Usart ( "%s", pStr );
89.         }
90.
91.     }

```

代码清单③: ESP8266_StaTcpClient_ApTcpServer 函数

```

1.  /*
2.  * 函数名: ESP8266_StaTcpClient_ApTcpServer
3.  * 描述   : PZ-ESP8266 模块进行 STA(TCP Client)+AP(TCP Server)测试
4.  * 输入   : 无
5.  * 返回   : 无
6.  * 调用   : 被外部调用
7.  */
8.  void ESP8266_StaTcpClient_ApTcpServer ( void )
9.  {
10.     char cStrInput [100] = { 0 }, * pStrDelimiter [3], * pBuf, * pStr;
11.     u8 uc = 0;
12.     u32 ul = 0;
13.
14.     ESP8266_Choose ( ENABLE );
15.
16.     ESP8266_AT_Test ();
17.
18.     ESP8266_Net_Mode_Choose ( STA_AP );

```

```

19.
20.
21.     PC_Usart ( "\r\n 请输入要创建的 WiFi 的名称、加密方式和密钥，加密方式的编号为： \
22.                                     \r\n0 = OPEN\
23.                                     \r\n1  =WEP\
24.                                     \r\n2 = WPA_PSK\
25.                                     \r\n3 = WPA2_PSK\
26.                                     \r\n4 = WPA_WPA2_PSK\
27.                                     \r\n 输入格式为： 名称字符+英文逗号+加密方式编号+英文逗号+密钥字符+空格，点
      击发送\r\n" );
28.
29.     scanf ( "%s", cStrInput );
30.
31.     PC_Usart ( "\r\n 稍等片刻 ..... \r\n" );
32.
33.     pBuf = cStrInput;
34.     uc = 0;
35.     while ( ( pStr = strtok ( pBuf, "," ) ) != NULL )
36.     {
37.         pStrDelimiter [ uc ++ ] = pStr;
38.         pBuf = NULL;
39.     }
40.
41.     ESP8266_BuildAP ( pStrDelimiter [0], pStrDelimiter [2], pStrDelimiter [1] );
42.     ESP8266_Cmd ( "AT+RST", "OK", "ready", 2500 ); /*
43.
44.
45.     ESP8266_Cmd ( "AT+CWLAP", "OK", 0, 5000 );
46.
47.     do
48.     {
49.         PC_Usart ( "\r\n 请输入要连接的 WiFi 名称和密钥，输入格式为： 名称字符+英文逗号+密钥字符+空格，
      点击发送\r\n" );
50.
51.         scanf ( "%s", cStrInput );
52.
53.         PC_Usart ( "\r\n 稍等片刻 ..... \r\n" );
54.
55.         pBuf = cStrInput;
56.         uc = 0;
57.         while ( ( pStr = strtok ( pBuf, "," ) ) != NULL )
58.         {
59.             pStrDelimiter [ uc ++ ] = pStr;
60.             pBuf = NULL;

```

```

61.         }
62.
63.     } while ( ! ESP8266_JoinAP ( pStrDelimiter [0], pStrDelimiter [1] ) );
64.
65.
66.     ESP8266_Enable_MultipleId ( ENABLE );
67.
68.
69.     PC_Usart ( "\r\n 请输入服务器要开启的端口号和超时时间 (0~28800, 单位: 秒), 输入格式为: 端口号
        字符+英文逗号+超时时间字符+空格, 点击发送\r\n" );
70.
71.     scanf ( "%s", cStrInput );
72.
73.     PC_Usart ( "\r\n 稍等片刻 ..... \r\n" );
74.
75.     pBuf = cStrInput;
76.     uc = 0;
77.     while ( ( pStr = strtok ( pBuf, "," ) ) != NULL )
78.     {
79.         pStrDelimiter [ uc ++ ] = pStr;
80.         pBuf = NULL;
81.     }
82.
83.     ESP8266_StartOrShutServer ( ENABLE, pStrDelimiter [0], pStrDelimiter [1] );
84.
85.
86.     do
87.     {
88.         PC_Usart ( "\r\n 请在电脑上将网络调试助手以 TCP Server 连接网络, 并输入电脑的 IP 和端口号, 输
            入格式为: 电脑 IP+英文逗号+端口号+空格, 点击发送\r\n" );
89.
90.         scanf ( "%s", cStrInput );
91.
92.         PC_Usart ( "\r\n 稍等片刻 ..... \r\n" );
93.
94.         pBuf = cStrInput;
95.         uc = 0;
96.         while ( ( pStr = strtok ( pBuf, "," ) ) != NULL )
97.         {
98.             pStrDelimiter [ uc ++ ] = pStr;
99.             pBuf = NULL;
100.        }
101.

```

```

102.     } while ( ! ( ESP8266_Link_Server ( enumTCP, pStrDelimiter [0], pStrDelimiter [1], Multiple_ID_0 ) &&
103.             ESP8266_Link_Server ( enumTCP, pStrDelimiter [0], pStrDelimiter [1], Multiple_ID_1 ) &&
104.             ESP8266_Link_Server ( enumTCP, pStrDelimiter [0], pStrDelimiter [1], Multiple_ID_2 ) ) );
105.
106.
107.     do
108.     {
109.         PC_Usart ( "\r\n 正查询本模块 IP, 前一个为 AP IP, 后一个为 STA IP.....\r\n" );
110.         ESP8266_Cmd ( "AT+CIFSR", "OK", "Link", 500 );
111.
112.         PC_Usart ( "\r\n 请用手机连接刚才创建的 WiFi, 这里只连接一个手机, 作为 ID3, 然后用手机网络调试助手以 TCP Client 连接刚才开启的服务器 (AP IP) ..... \r\n" );
113.         delay_ms ( 20000 );
114.     } while ( ! ESP8266_Cmd ( "AT+CIPSTATUS", "+CIPSTATUS:3", 0, 500 ) );
115.
116.
117.     for ( uc = 0; uc < 3; uc ++ )
118.     {
119.         PC_Usart ( "\r\n 请输入端口 ID%d 要发送的字符串, 输入格式为: 字符串 (不含空格)+空格, 点击发送\r\n", uc );
120.
121.         scanf ( "%s", cStrInput );
122.
123.         ul = strlen ( cStrInput );
124.
125.         ESP8266_SendString ( DISABLE, cStrInput, ul, ( ENUM_ID_NO_TypeDef ) uc );
126.
127.     }
128.
129.
130.     PC_Usart ( "\r\n 请输入要向端口手机 (ID3) 发送的字符串, 输入格式为: 字符串 (不含空格)+空格, 点击发送\r\n" );
131.
132.     scanf ( "%s", cStrInput );
133.
134.     ul = strlen ( cStrInput );
135.
136.     ESP8266_SendString ( DISABLE, cStrInput, ul, Multiple_ID_3 );
137.
138.
139.     PC_Usart ( "\r\n 请在电脑网络调试助手或手机网络调试助手发送字符串.....\r\n" );

```



```

140.     while (1)
141.     {
142.         pStr = ESP8266_ReceiveString ( DISABLE );
143.         PC_Usart ( "%s", pStr );
144.     }
145.
146. }

```

在 `wifi_function.c` 文件定义了 PZ-ESP8266 WiFi 模块的配置和使用子函数，且均说明了函数的使用情况，用户可以阅览该源文件了解详细内容，这里不再赘述。

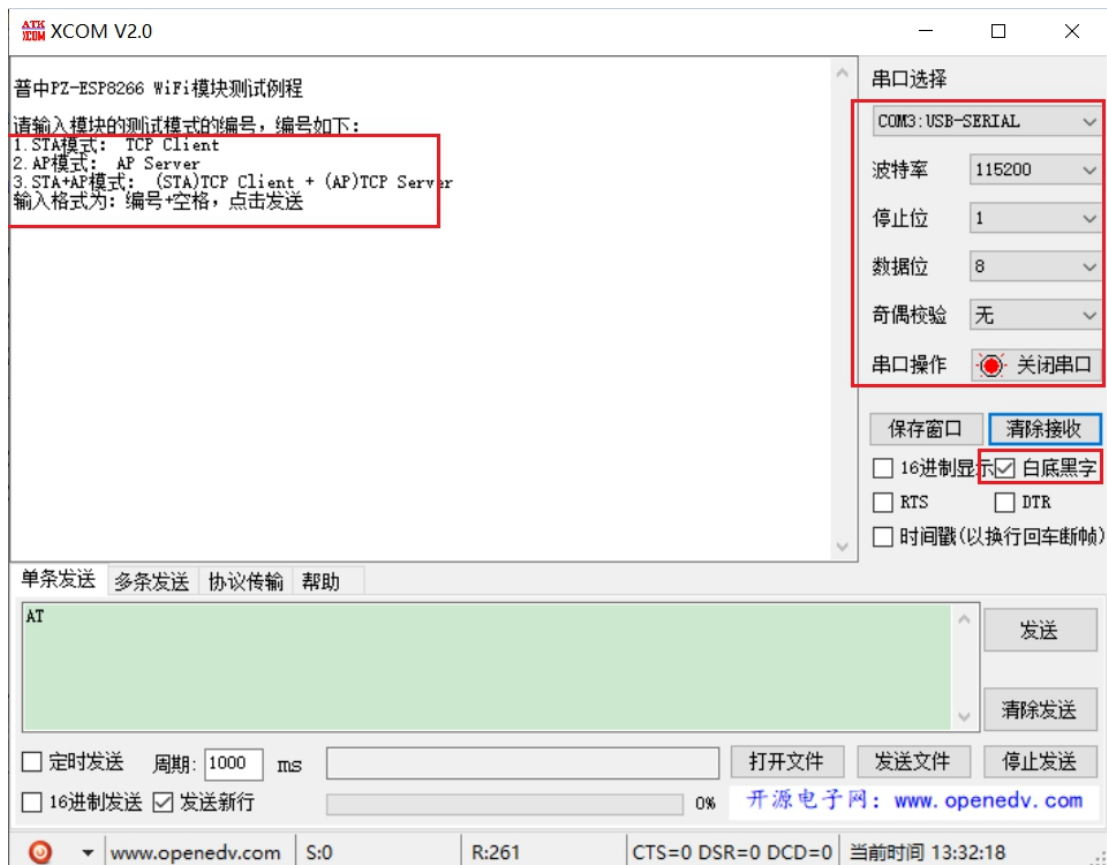
3.3 实验现象

将 PZ-ESP8266 模块插在天马&麒麟开发板上的 WIFI 接口处，根据板子上丝印插入，注意方向，注意开发板上 USB 转 TTL 模块的 P4 端子短接片要连接好，如下：

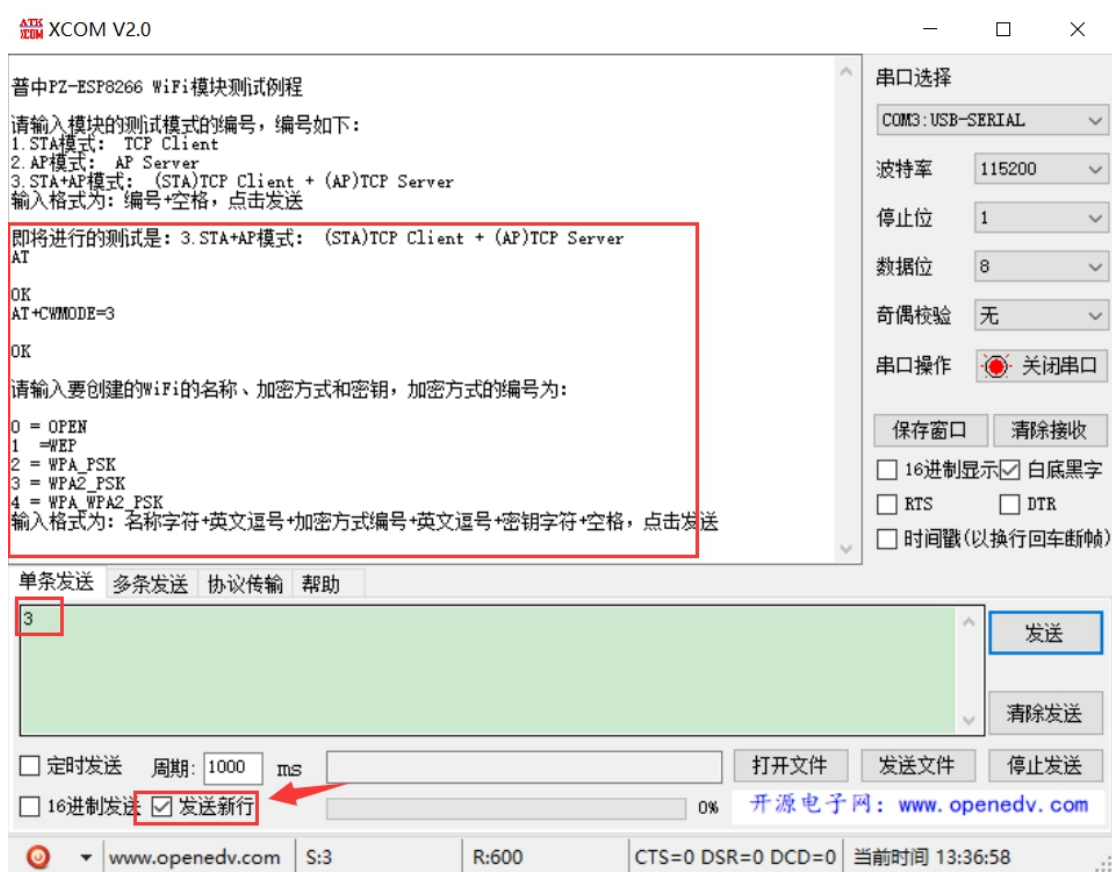


如果使用的 STM32 开发板不含 WIFI 接口，请按照前面“硬件连接”章节连接好。

(1) 将 USB 线与开发板和电脑连接，打开电源开关。打开串口调试助手“**XCOM V2.0.exe**”，实验过程中用户只需要按照串口调试助手打印的信息进行操作即可，以下是我在测试过程中的实验现象的部分截图，供用户参考。

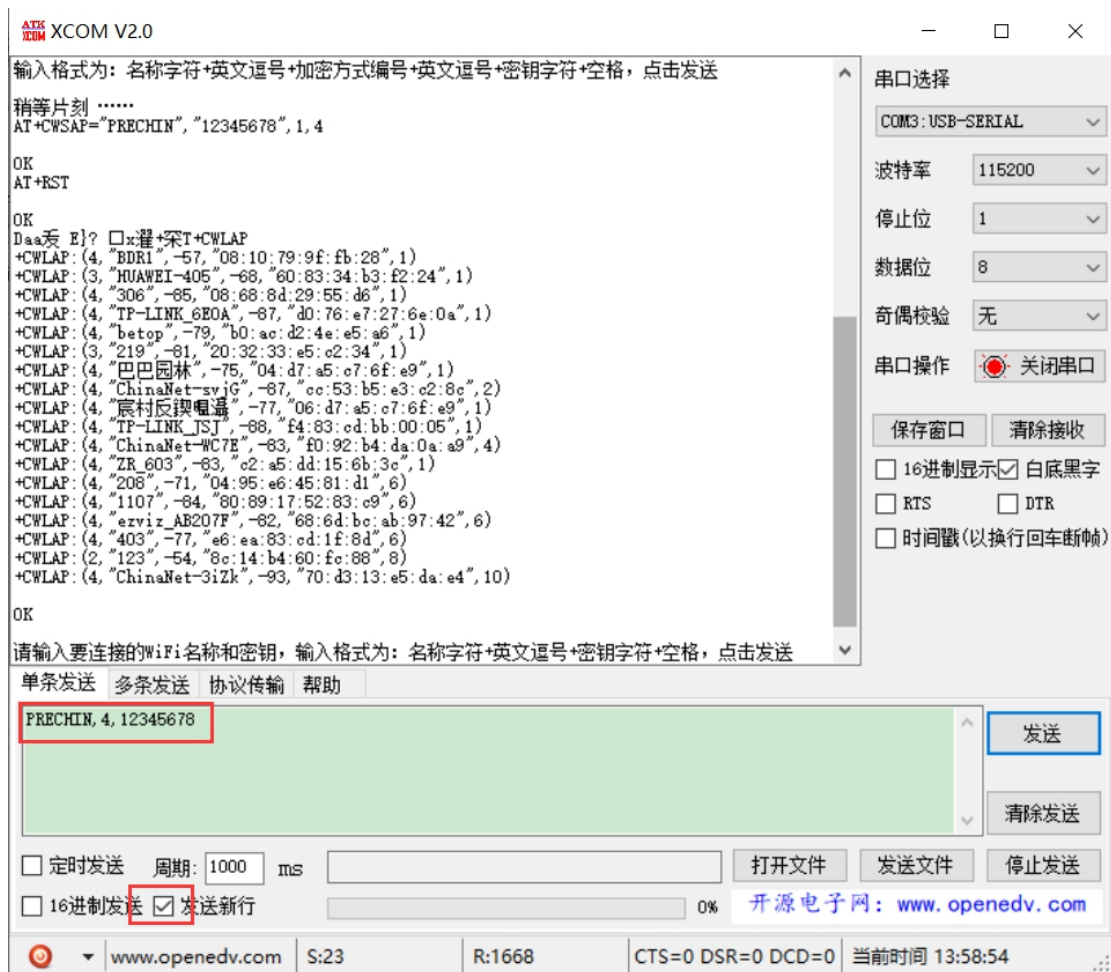


(2) 从上图可知，根据输出的结果，选择对应的测试编号，这里我选择编号“3”，既可对模块的 STA 模式测试，又可对 AP 模式测试。因此，在输入对话框中输入：3，注意，要勾选“发送新行”。如下：

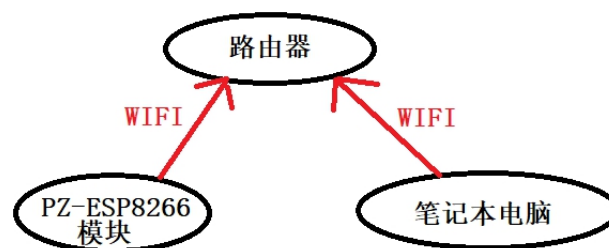


(3) 按照输出提示信息，需要给模块创建 WIFI 热点名称、加密方式和密钥，
按照规定格式输入：PRECHIN, 4, 12345678

串口输出信息如下：

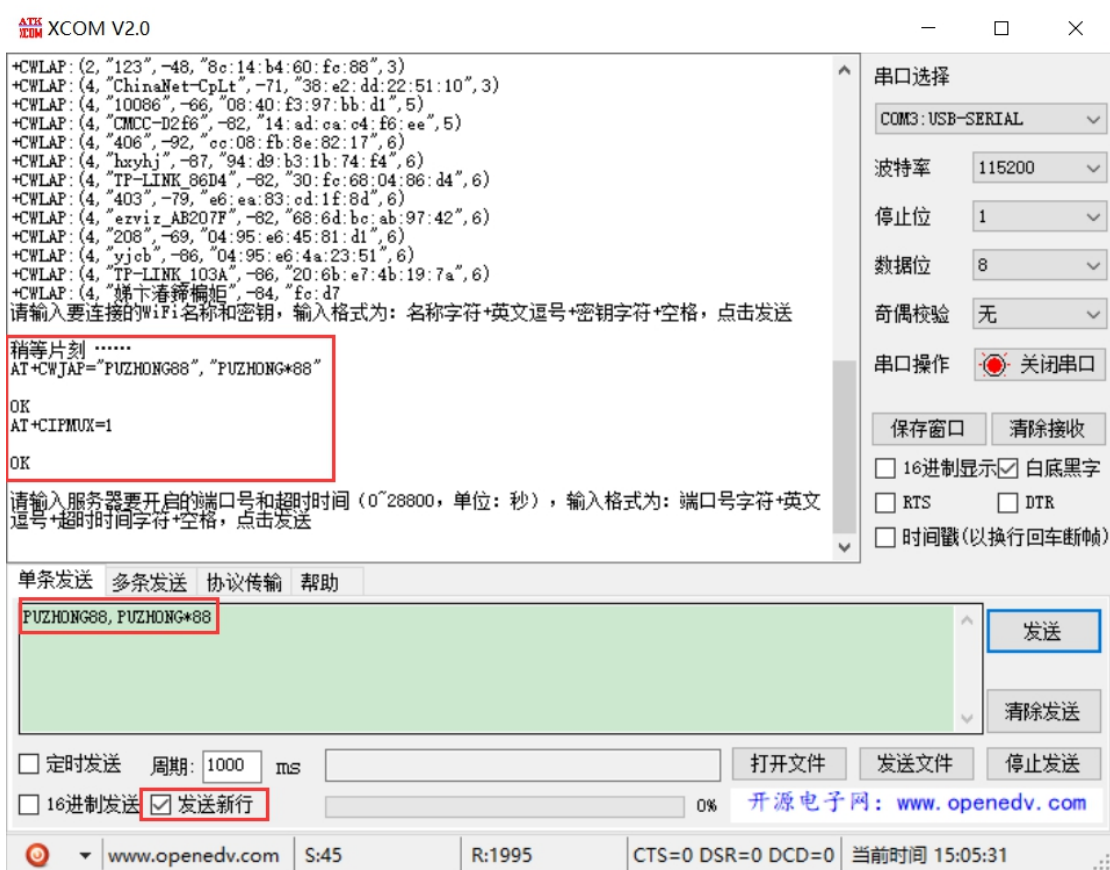


(4) 接着串口助手又输出提示信息，需要我们输入模块要连接的 WIFI 名称和密码，比如路由器产生一个 WIFI 热点，然后模块去连接这个热点，并且保证电脑也是连接在路由器下的网络，这样 WIFI 模块即可通过路由器与电脑实现网络通信。我测试时采用如下连接方式：



按照指定的格式输入：PUZHONG88,PUZHONG*88（这里根据用户自己的路由器 WIFI 热点自行设置）

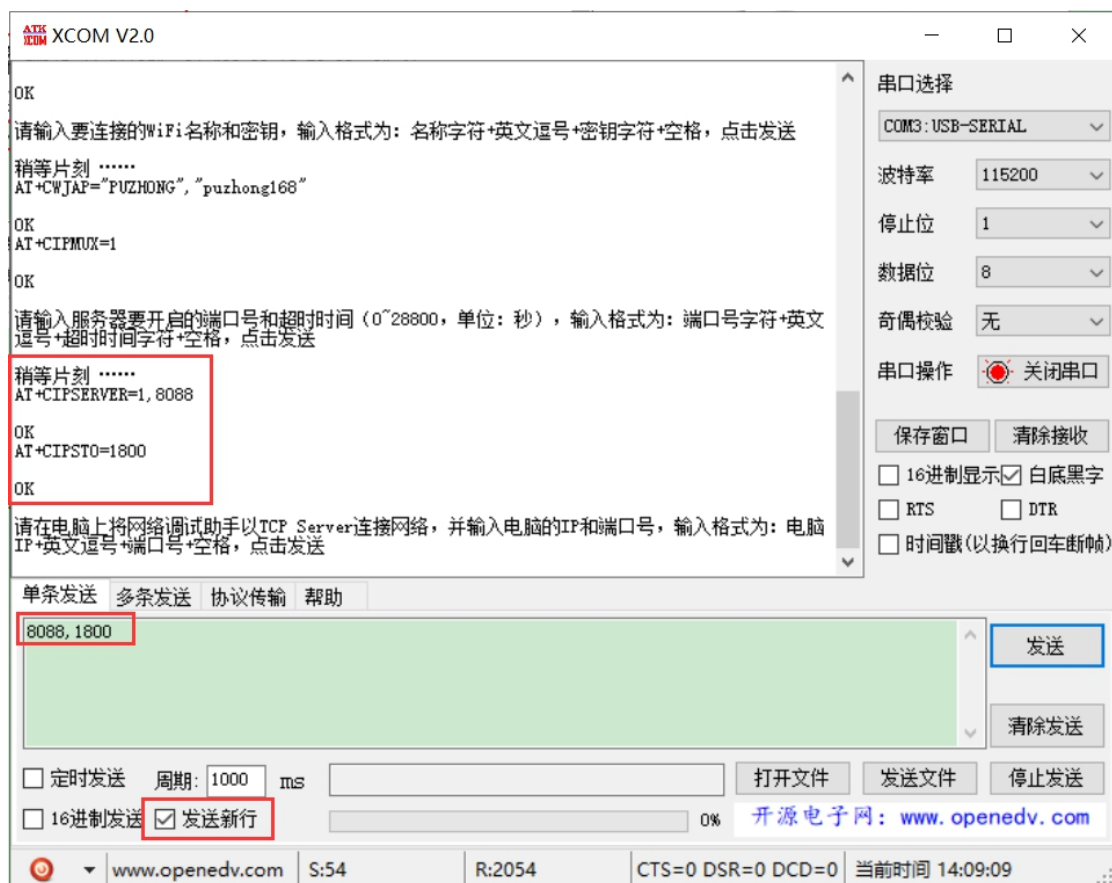
串口输出如下：



若出现连接失败或超时，请再次发送该命令。

(5) 接着串口助手又输出提示信息，需要我们输入服务器要开启的端口号和超时时间，按照指定的格式输入：8088, 1800

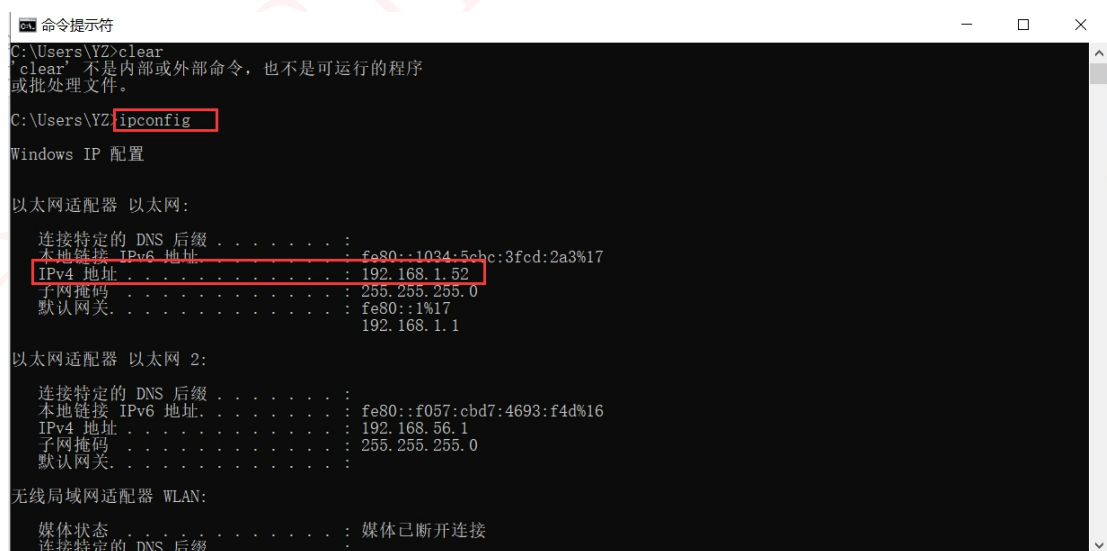
串口输出信息如下：



(6) 接着串口助手又输出提示信息，需要我们输入电脑的 IP 和端口号，在串口助手软件输入之前，我们先在电脑端打开网络调试助手，并设置电脑 IP 和端口号，网络调试助手设置如下：

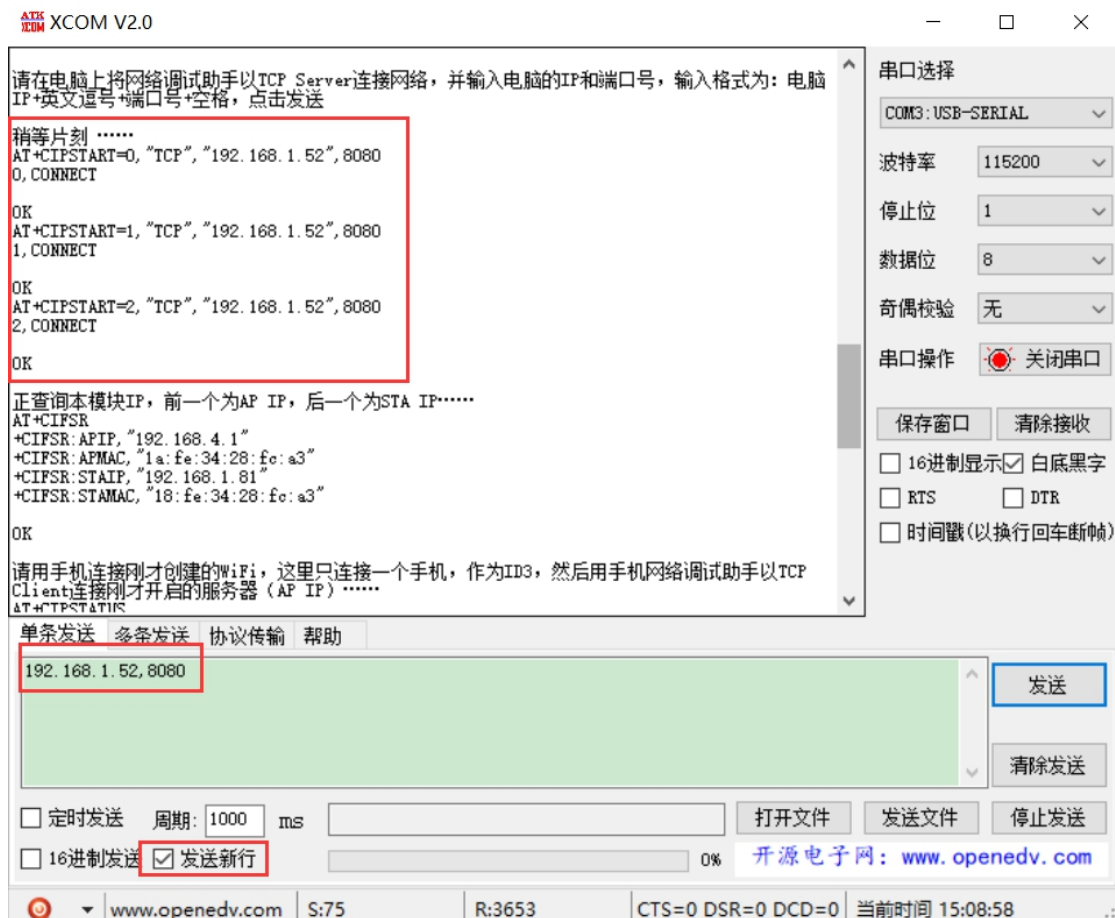


如果不知道自己电脑 IP 地址的，可以直接在命令窗口输入命令：ipconfig

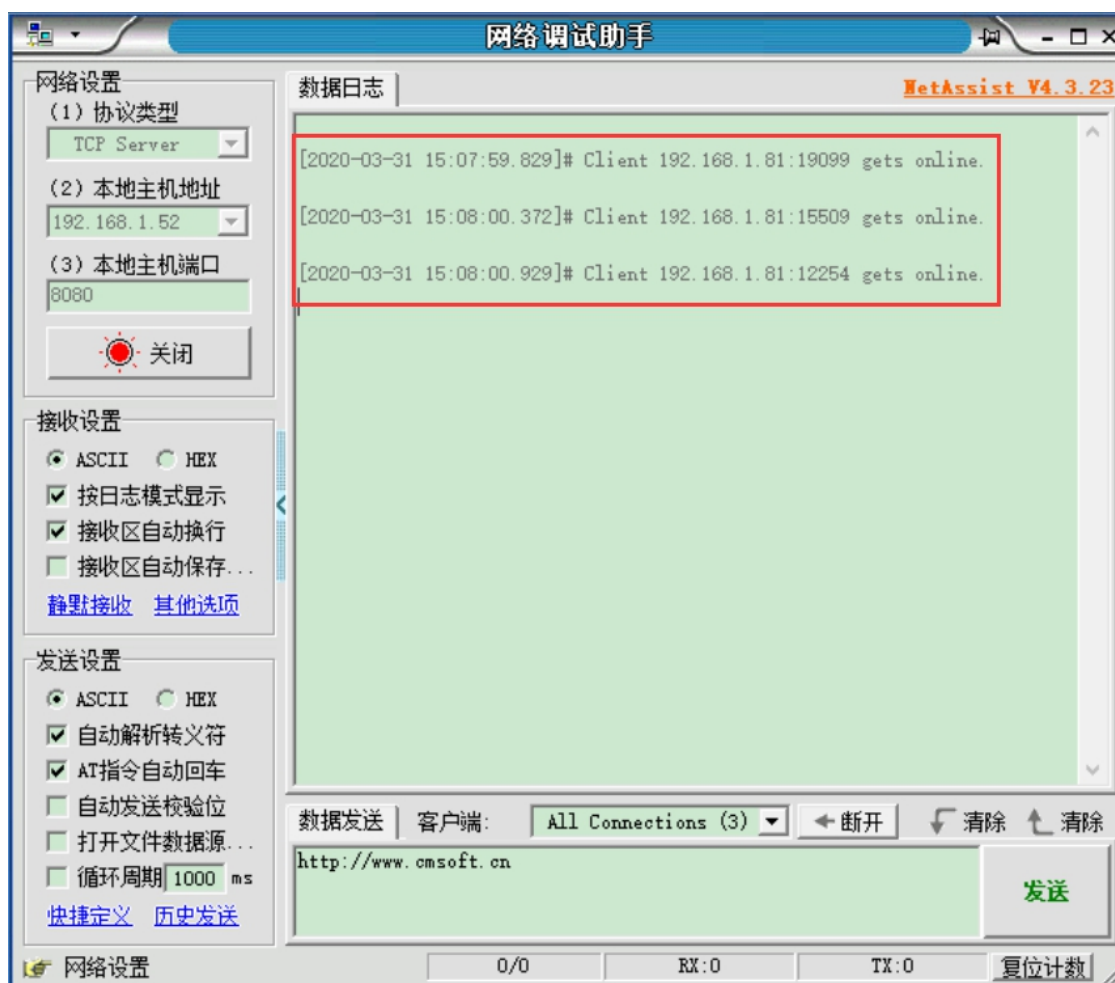


然后，回到串口助手，按照指定的格式输入：192.168.1.52,8080

串口输出信息如下：



串口输出信息后，同时在网络调试助手上也会有信息输出，如下：



(7) 上一步操作完成后，其实串口最后又输出提示信息，需要我们用手机连接之前给模块创建的热点“PRECHIN”，并且前面我们说了，PZ-ESP8266 模块可同时多个手机连接热点，本实验只演示一个手机连接，并且其连接的 ID 是 ID3。

首先需要给手机安装一个网络调试助手 APP，在模块资料“..\调试工具\手机端网络调试助手\TCP.apk”中我们已经提供，在安卓手机上安装该 APP 即可。然后用手机连接前面给模块设置的 WIFI 热点：PRECHIN，密码是 12345678。如下所示：



打开安装好的 APP，首先要新建一个主机，IP 地址设置为：192.168.4.1，端口设置为：8088。操作如下：



手机连接成功后，串口输出如下，因为例程中手机连接作为 ID3，所以前面的 ID0、ID1、ID2 我们需要按照提示发送，以实现 ID3 的连接，这样手机 APP 即可与模块实现数据收发，网络助手和模块即可实现数据收发。

ATX
IDM

XCOM V2.0

OK
3, CONNECT

正查询本模块IP, 前一个为AP IP, 后一个为STA IP.....
AT+CIFSR
+CIFSR:APIP,"192.168.4.1"
+CIFSR:AFMAC,"1a:fe:34:28:fe:a3"
+CIFSR:STAPIP,"192.168.1.81"
+CIFSR:STAMAC,"18:fe:34:28:fe:a3"

OK

请用手机连接刚才创建的WiFi, 这里只连接一个手机, 作为ID3, 然后用手机网络调试助手以TCP
Client连接刚才开启的服务器 (AP IP).....
AT+CIPSTATUS
STATUS:2
+CIPSTATUS:0,"TCP","192.168.1.52",8080,0
+CIPSTATUS:1,"TCP","192.168.1.52",8080,0
+CIPSTATUS:2,"TCP","192.168.1.52",8080,0
+CIPSTATUS:3,"TCP","192.168.4.4",37689,1

OK

请输入端口ID0要发送的字符串, 输入格式为: 字符串 (不含空格) *空格, 点击发送
AT+CIPSEND=0,5
> ID0
请输入端口ID1要发送的字符串, 输入格式为: 字符串 (不含空格) *空格, 点击发送

串口选择
COM3: USB-SERIAL

波特率 115200

停止位 1

数据位 8

奇偶校验 无

串口操作  关闭串口

保存窗口 清除接收

☐ 16进制显示 ☒ 白底黑字

☐ RTS ☐ DTR

☐ 时间戳 (以换行回车断帧)

单条发送 多条发送 协议传输 帮助

ID0

发送ID0

发送

清除发送

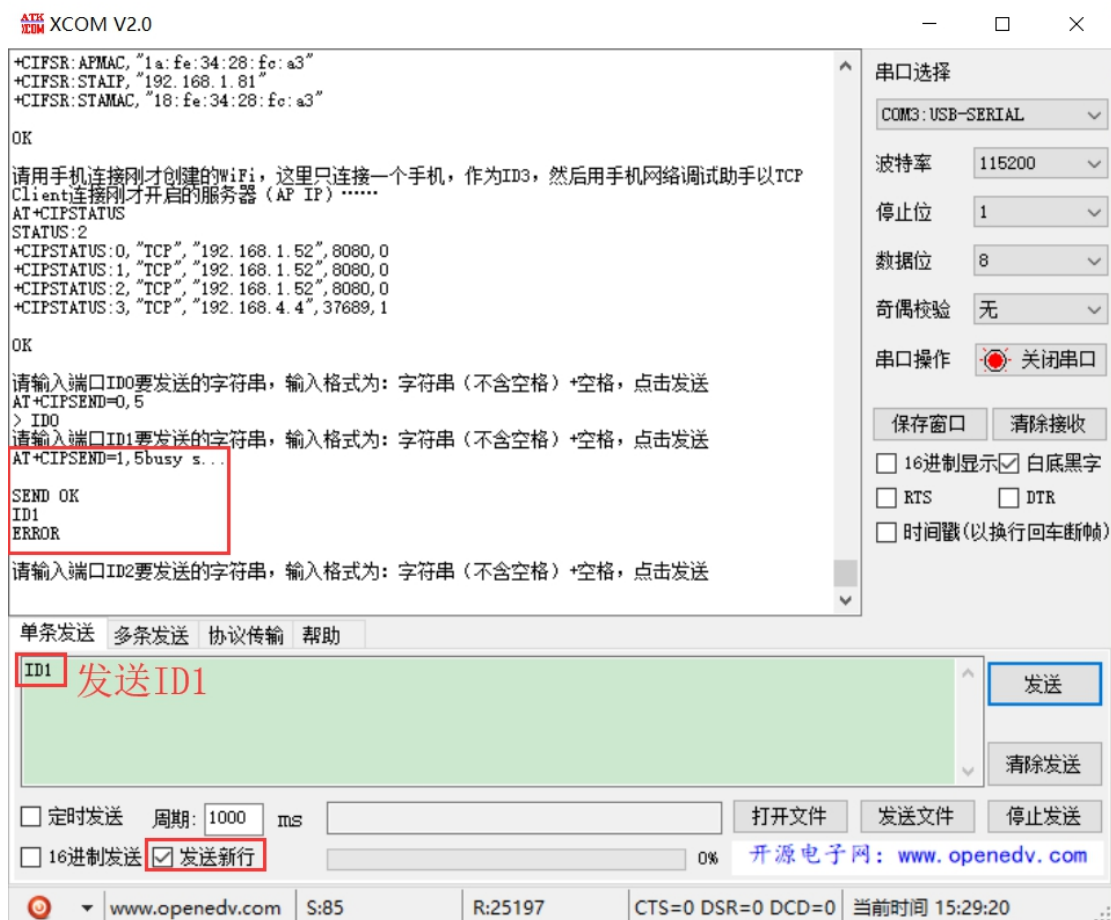
☐ 定时发送 周期: 1000 ms

☐ 16进制发送 ☒ 发送新行

打开文件 发送文件 停止发送

开源电子网: www.openedv.com

www.openedv.com S:80 R:25071 CTS=0 DSR=0 DCD=0 当前时间 15:25:04



OK

用手机连接刚才创建的WiFi，这里只连接一个手机，作为ID3，然后用手机网络调试助手以TCP Client连接刚才开启的服务器（AP IP）.....

AT+CIPSTATUS

STATUS:2

+CIPSTATUS:0,"TCP","192.168.1.52",8080,0

+CIPSTATUS:1,"TCP","192.168.1.52",8080,0

+CIPSTATUS:2,"TCP","192.168.1.52",8080,0

+CIPSTATUS:3,"TCP","192.168.4.4",37689,1

OK

请输入端口ID0要发送的字符串，输入格式为：字符串（不含空格）*空格，点击发送

AT+CIPSEND=0,5

> ID0

请输入端口ID1要发送的字符串，输入格式为：字符串（不含空格）*空格，点击发送

AT+CIPSEND=1,5busy s...

SEND OK

ID1

ERROR

请输入端口ID2要发送的字符串，输入格式为：字符串（不含空格）*空格，点击发送

AT+CIPSEND=2,5

> ID2

请输入要问端口手机（ID3）发送的字符串，输入格式为：字符串（不含空格）*空格，点击发送

串口选择

COM3:USB-SERIAL

波特率115200

停止位1

数据位8

奇偶校验无

串口操作

关闭串口

保存窗口

清除接收

☐ 16进制显示
☒ 白底黑字

☐ RTS
☐ DTR

☐ 时间戳(以换行回车断帧)

单条发送 多条发送 协议传输 帮助

ID2

发送ID2

发送

清除发送

☐ 定时发送 周期:1000 ms

☐ 16进制发送
☒ 发送新行

打开文件

发送文件

停止发送

0%

开源电子网: www.openedv.com

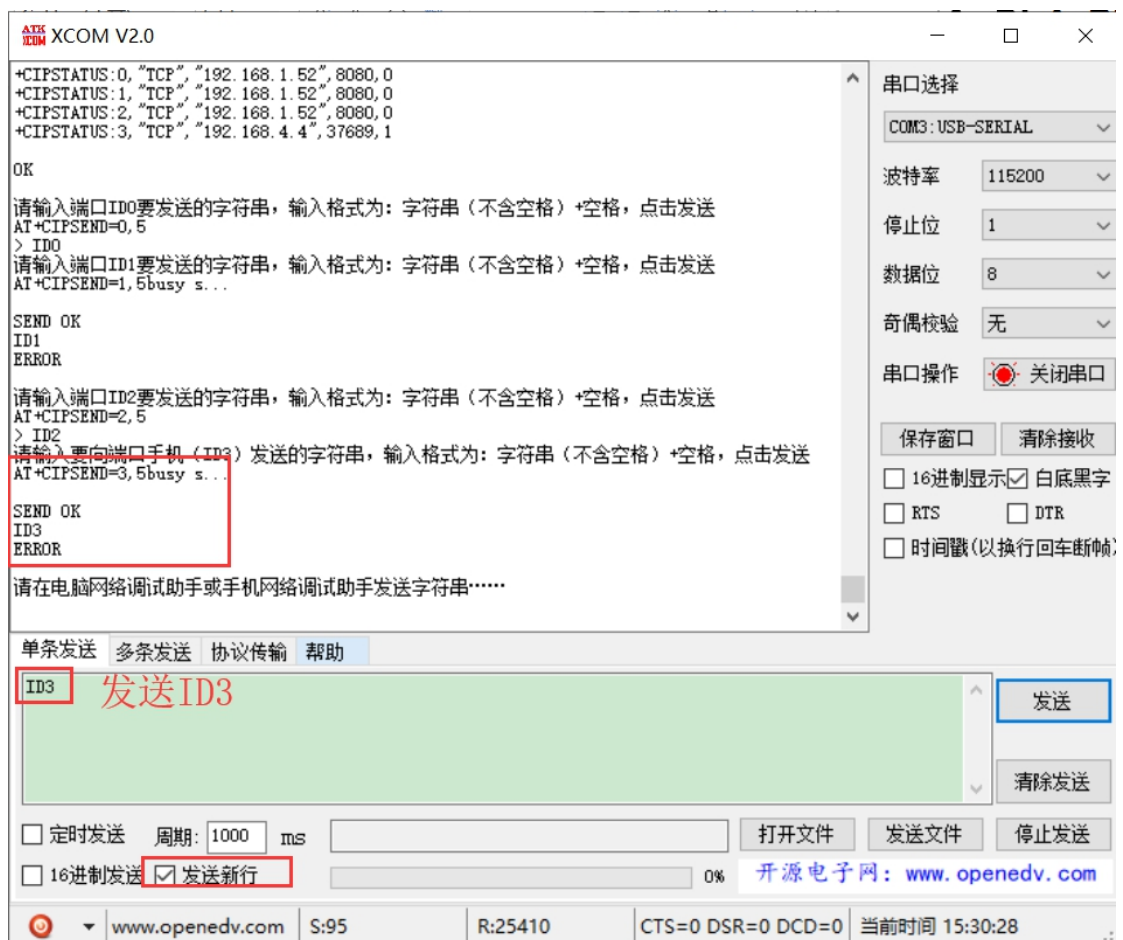
www.openedv.com

S:90

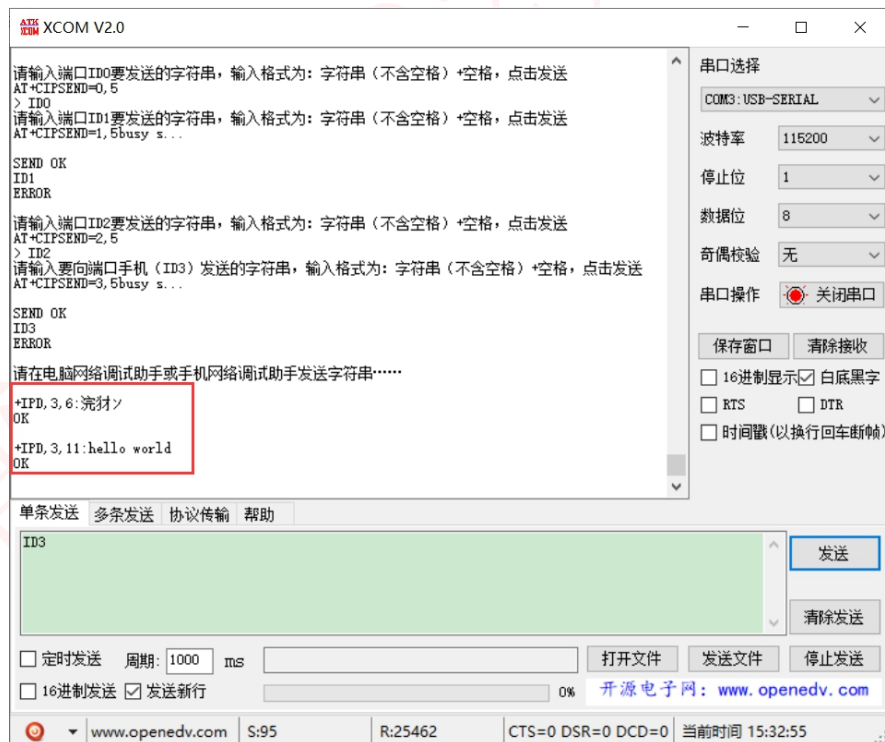
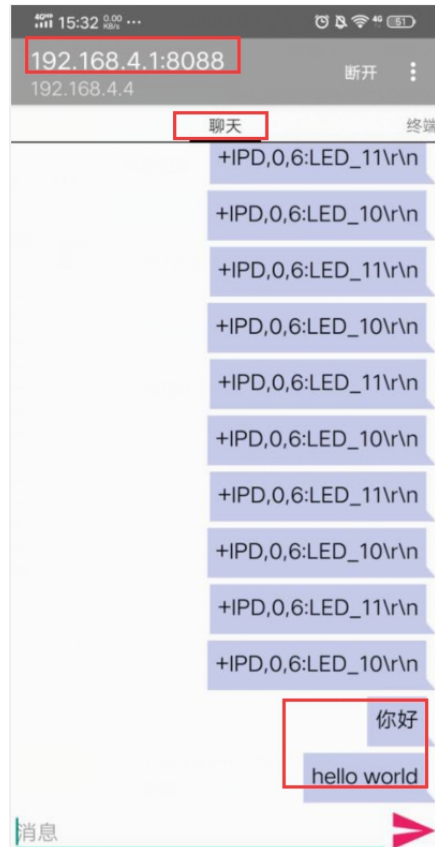
R:25306

CTS=0 DSR=0 DCD=0

当前时间 15:29:53



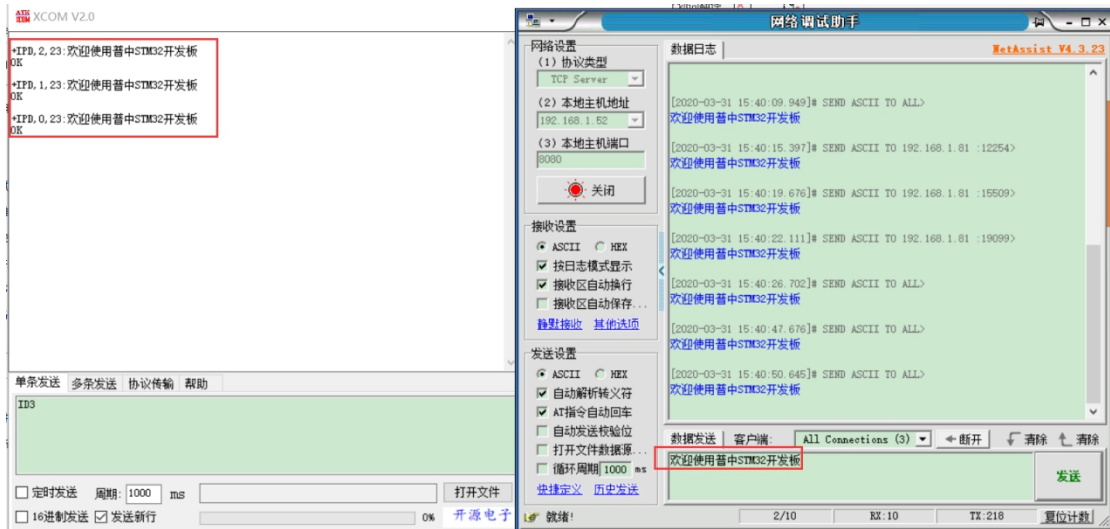
接下来就可以实现电脑网络助手或手机网络助手实现数据收发了。



在手机 APP 中，我们发送了一条“你好”和一条“hello world”，在串口助手上也分别显示出来，其中你好显示乱码，这个可能是串口助手中文解码问题。细心的朋友可能会发现，明明我发送的数据只有 hello world，怎么前面还显示

了“+IPD, 3, 11:”，这是因为模块固件里面自带发送对应的 ID 和消息长度，+IPD 是固定的，3 表示此时发送数据的 ID 是 ID3，11 表示数据长度。

我们还可以在电脑端网络调试助手上发送数据给模块，模块可通过串口打印输出。网络调试助手现在选择的客户端有 3 个，分别是 ID0、ID1、ID2，我们手机连接了 ID3。只要在网络助手上发送数据，可同时发给 ID0、ID1 和 ID2 的客户端。



4 PZ-ESP8266 模块固件升级

PZ-ESP8266 WiFi 模块升级固件的方法有两种：第一种是本地串口烧录，第二种是云端升级。笔者着重介绍第一种方法，第二种方法用户可以参考第三方资料的相关手册。

4.1 本地串口烧录

使用“本地串口烧录”方案刷新固件的步骤如下：

(1) 断开模块电源，按照下列方式连接好电路。GPIO0 引脚拉低进入 FLASH 烧写模式。（事先将开发板上 USB 转 TTL 模块 P4 端子的黄色跳线帽取下）

（PZ-ESP8266 模块-->STM32 开发板）

VCC-->3.3V

CH_PD-->3.3V

TXD-->URXD（开发板上 USB 转 TTL 模块）

RXD-->UTXD（开发板上 USB 转 TTL 模块）

GND-->GND

I00-->GND

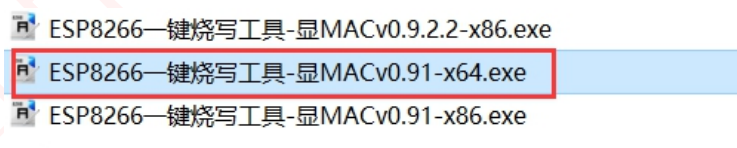
（2）上网下载固件。网上有很多技术论坛都有制作 ESP8266 芯片固件，用户可以自行上网搜索，根据自己的需求下载中意的固件。笔者要演示的固件是从“物联世界”论坛下载的安信可官方的 0.95 版本的固件，存放于“..\ESP8266 固件及烧写工具\ESP8266 固件\安信可官方 0.95\v0.9.5.0 AT Firmware.bin”。

（3）选择符合电脑系统的“ESP8266 一键烧写工具”。在电脑上右键单击“计算机”图标选择“属性”查看电脑系统类型。如下：

查看有关计算机的基本信息



打开“..\ESP8266 固件及烧写工具\ESP8266 一键烧写工具\ESP8266 一键烧写工具”，根据电脑系统类型选择“ESP8266 一键烧写工具”，打开烧写工具，如下图。





(4) 烧录固件。点击“ESP8266 一键烧写工具”的“配置”选项卡，点击第二栏的图标，选择固件的路径（“..\ESP8266 固件及烧写工具\ESP8266 固件\安信可官方 0.95\v0.9.5.0 AT Firmware.bin”），仅将第二栏打“x”，如下图所示：



按照前面接线方式，给模块上电。点击“操作”选项卡，选择连接到模块的串口，点击“一键烧写(F)”即可进行烧写固件。烧写过程中有进度条，而且还能读取模块的物理地址，如下图所示。



烧写完毕后，工具左下角会出现一个图标，如下图所示。



5 其他

(1) 购买地址（普中授权店铺）

<http://www.prechin.net/forum.php?mod=viewthread&tid=38746&extra=>

(2) 资料下载

<http://prechin.net/forum.php?mod=viewthread&tid=35264&extra=page%3D1>

(3) 技术支持

普中官网: www.prechin.cn

普中论坛: www.prechin.net

技术电话: 0755-21509063 (转技术)

普中STM32开发板